

Capítulo 4: Tornando o Site Dinâmico com ViewBag e Razor

No mundo real, um site precisa exibir informações que mudam o tempo todo: o nome do usuário logado, a data de hoje, ou uma lista de produtos vinda de um banco de dados.

Para fazer isso no ASP.NET Core, usamos duas ferramentas principais: o ViewBag (a "mochila" de dados) e o Razor (o tradutor do C# para a tela).

O que é a ViewBag?

Imagine que a ViewBag é uma mochila que o Controller entrega para a View antes de a página ser carregada. Você pode colocar qualquer coisa dentro dessa mochila (textos, números, listas) lá no C#, e a View pode abrir essa mochila e mostrar o conteúdo na tela.

Passo a Passo: Enviando uma mensagem

Vamos fazer o HomeController enviar uma mensagem de boas-vindas personalizada para a página inicial.

1. Abra o arquivo Controllers/HomeController.cs.
2. Vá até o método Index() e adicione uma informação dentro da ViewBag, logo antes do return View();:

```
public IActionResult Index()
{
    // Colocando uma informação na "mochila"
    ViewBag.MensagemBemVindo = "Olá, futuros desenvolvedores!";
    ViewBag.HoraAtual = DateTime.Now.ToString("HH:mm");

    return View();
}
```

3. Salve o arquivo. O Controller já fez a parte dele. Agora precisamos fazer a View mostrar isso.

O que é o Razor (@)?

O HTML normal não entende C#. Para misturar os dois, o ASP.NET usa o Razor. A regra é simples: toda vez que você digita o símbolo @ no HTML, o arquivo entende que a próxima palavra ou bloco é um código C#.

1. Abra o arquivo Views/Home/Index.cshtml.
2. Apague o texto antigo e use o símbolo @ para "puxar" as informações da sua ViewBag:

```
<div class="text-center">
  <h1>@ViewBag.MensagemBemVindo</h1>
  <p>O horário atual no servidor é: <strong>@ViewBag.HoraAtual</strong></p>
</div>
```

3. Pare o servidor (Ctrl + C) e rode novamente com dotnet run. Ao abrir a página inicial, você verá as mensagens geradas pelo C# em tempo real!

O Poder do Razor: Lógica de Programação no HTML

O Razor não serve apenas para mostrar variáveis. Você pode usar comandos do C# (como if e foreach) direto na tela!

Vamos criar uma lista de recados. No arquivo Views/Home/Index.cshtml, adicione o seguinte código no final da página:

```
<hr />
<h3>Avisos Importantes:</h3>

@{
    // Este bloco @{ ... } permite escrever várias linhas de C# dentro da View
    int quantidadeAulas = 4;
    bool temProvaHoje = false;
}

@if(temProvaHoje)
{
```

```
<div class="alert alert-danger">
  Atenção: Hoje tem prova de programação!
</div>
}
else
{
  <div class="alert alert-success">
    Ufa! Sem provas hoje. Foco no projeto!
  </div>
}

<p>Faltam @quantidadeAulas aulas para terminar o módulo.</p>
```

Teste no navegador! Mude a variável temProvaHoje para true, salve e atualize a página. A cor e o aviso vão mudar sozinhos.

Exercícios Práticos para a Turma

Agora é a hora de treinar essa comunicação entre o Back-end e o Front-end.

Exercício 1: Cumprimento Inteligente (Na página Sobre)

- Abram o HomeController.cs e vão até a ação Sobre().
- Criem uma ViewBag.NomeEscola = "Etec"; e uma ViewBag.AnoFundacao = 2006; (ou o ano correto!).
- Abram a view Sobre.cshtml e escrevam um parágrafo que utilize essas variáveis usando o @ViewBag. Por exemplo: "A @ViewBag.NomeEscola forma excelentes profissionais desde @ViewBag.AnoFundacao."

Exercício 2: O Desafio do foreach (Na página Cursos)

Vamos listar cursos usando um laço de repetição (foreach), para não precisarmos digitar cada na mão.

- Vão até a view Cursos.cshtml que vocês criaram no capítulo passado.
- Apaguem a lista feita em HTML puro.
- Criem um bloco de código Razor e declarem uma lista de strings com os cursos:

```
@{
    List<string> listaCursos = new List<string>()
    {
        "Desenvolvimento de Sistemas",
        "Eletrônica",
        "Administração",
        "Mecatrônica"
    };
}
```

- Logo abaixo do bloco, utilizem um @foreach para desenhar a lista no HTML:

```
<ul>
    @foreach(string curso in listaCursos)
    {
        <li>@curso</li>
    }
</ul>
```

Testem no navegador! Adicionem um novo curso à lista C# no topo do arquivo, salvem, e vejam a mágica acontecer: o HTML vai se adaptar e gerar um novo item na lista automaticamente.